

Presentation Script

AI-Based Adaptive Security Gateway for SQL Injection and XSS Detection

BCY586 Mini Project — Review 1 | 20-minute slot, ~17 slides, ~70 seconds/slide

1. Title Slide

SLIDE 1 — Title

Good morning/afternoon everyone. We are presenting our mini project titled 'AI-Based Adaptive Security Gateway for SQL Injection and XSS Detection', under the guidance of [Guide Name]. I am [Team Lead Name], and this is my teammate [Team Member 2 Name]. Our project is from the Web Security and Artificial Intelligence domain, and today we'll walk you through the problem we're solving, our objectives, the literature that shaped our approach, our methodology, and what we expect to deliver.

Presenter tip: Keep this to 15-20 seconds — just introduce the team and title, don't start explaining content yet.

SLIDE 2 — Tribute to Our Founder

Before we begin, we pay tribute to our founder, Dr. R N Shetty, whose vision continues to guide this institution.

Presenter tip: This is a standard institutional slide — a brief, respectful pause, no more than 10 seconds.

SLIDE 3 — Agenda

Here's how we'll structure today's presentation. We'll start with an introduction to the problem space, then cover our problem statement and objectives, walk through five research papers in our literature review, explain our methodology including the complete system architecture, list the resources and requirements, and close with expected outcomes, real-world applications, and our references.

2. Introduction

SLIDE 4 — Introduction — Background & Motivation

Web applications today are central to almost every digital service we use — banking, healthcare records, communication platforms. Because of this, they've become high-value targets for attackers. Two of the most persistent threats are SQL Injection and Cross-Site Scripting, both of which have appeared in the OWASP Top 10 list of web application security risks for over a decade. SQL Injection works by manipulating a database query through malicious input — for example, tricking a login form into returning data it shouldn't. Cross-Site Scripting, on the other hand, injects malicious scripts into a page that other users will later view, which can be used to steal session cookies or redirect users to fake sites. Despite being very different in mechanism, both attacks share the same root cause: insufficient validation and sanitization of user-supplied input.

SLIDE 5 — Introduction (contd.) — The Gap in Current Defenses

Now, how do most existing systems defend against this today? They rely heavily on signature-based or regular expression filtering — essentially, a list of known bad patterns that get blocked. The problem is, this approach is inherently brittle. It can only catch attacks that someone has already anticipated and written a rule for. Attackers routinely bypass these filters using encoding tricks, case variations, or simply crafting a new payload structure that doesn't match any existing signature. Even more critically, these regex-based systems never improve on their own — every time a new bypass technique is discovered, a human has to manually go in and rewrite the rules. This is the gap our project addresses.

SLIDE 6 — Introduction (contd.) — Our Approach

Our solution is what we call an Adaptive Security Gateway — a system where the detection core is a deep learning model instead of static rules. Specifically, we use a character-level CNN/BiLSTM neural network, built in PyTorch and trained on a CUDA-enabled GPU, which learns attack patterns directly from raw payload text rather than relying on anyone manually writing detection rules. But our project is more than just a classifier — it's architected as a complete security gateway. This means it also handles request parsing, threat intelligence enrichment, session and device tracking, behavioral anomaly scoring, policy enforcement, incident logging, and gives security analysts a dashboard to review flagged activity. Our core technology stack is Flask for the backend, PyTorch for the model, React for the dashboard, and MongoDB or PostgreSQL for data storage.

3. Problem Statement and Objectives

SLIDE 7 — Problem Statement and Project Objectives

Let me state our problem clearly: existing signature and regex-based input validation mechanisms fail to reliably detect obfuscated or previously unseen SQL Injection and Cross-Site Scripting payloads. Our project addresses this gap by embedding a deep learning-based detection core inside a complete security gateway pipeline. To achieve this, we've set five concrete objectives. First, to design and train a deep learning model — specifically a CNN/BiLSTM — using PyTorch on a CUDA-enabled GPU, to classify incoming input as benign, SQL Injection, or Cross-Site Scripting. Second, to build the complete gateway pipeline around this model — covering request parsing, preprocessing, threat intelligence, session management, device profiling, and behavioral anomaly scoring. Third, to design a risk-scoring and decision framework that produces one of four verdicts — Allow, Monitor, Rate Limit, or Block — with policy overrides for especially critical attacks. Fourth, to implement incident logging and build an analyst-facing dashboard where a security analyst can review and override these verdicts. And fifth, to evaluate how reliable our detection is using a held-out test set as well as payloads the model has never seen before.

4. Literature Review

SLIDE 8 — Literature Review (1/3)

We reviewed five papers to ground our approach. The first, by Angula and Hashiyana, compared several classical machine learning classifiers — Logistic Regression, Naive Bayes, Decision Tree, Random Forest, KNN, and SVM — for SQL Injection detection, and found Naive Bayes achieved the highest accuracy among them. This paper is important to us because it establishes the classical machine learning accuracy baseline that our deep learning model is ultimately measured against. The second paper, by Aburashed, AL Amoush and Alrefai from 2024, evaluated Random Forest, Gradient Boosting, SVM, and an Artificial Neural Network, and found the ANN achieved the best overall performance. This result directly supports our decision to move away from classical models and use a neural network as our detection core instead.

SLIDE 9 — Literature Review (contd.) (2/3)

The third paper, by Krishna, Oluoch and Kim from 2025, proposed a hybrid pipeline that combines Naive Bayes, LSTM-based feature extraction, and a Random Forest classifier. The use of an LSTM component here directly supports our own choice of a CNN/BiLSTM architecture as our primary detector — it confirms that sequence-aware neural models pick up patterns in attack payloads that simpler feature-based methods tend to miss. The fourth paper, published in Scientific Reports in 2024, applies a deep learning approach using the HTTP CSIC 2010 dataset, which contains both normal and attack-based HTTP requests covering SQL Injection, XSS, and other categories. This is probably the most directly relevant prior work for us — it confirms deep learning is an established, effective approach for exactly this problem, and validates that detecting at the full-request level, not just isolated payloads, is a sound design choice.

SLIDE 10 — Literature Review (contd.) (3/3)

The fifth paper, UniEmbed, combines multiple text-embedding techniques — Word2Vec, Universal Sentence Encoder, and FastText — as input features for a Multi-Layer Perceptron classifier detecting both XSS and SQL Injection together. This supports our decision to build a single, unified deep learning model that handles both attack types in one go, rather than training two separate binary classifiers. Taken together, these five papers reveal a gap in the existing literature: most prior work is either purely about the classifier itself, or doesn't integrate detection into a complete gateway pipeline with risk scoring, policy enforcement, and an analyst dashboard. That gap is exactly what our project addresses — a deep learning detection core, embedded inside a full fourteen-stage adaptive security gateway.

5. Methodology

SLIDE 11 — Methodology — Overview and Block Diagram

Here is the complete architecture of our gateway. Every incoming request passes through a fourteen-stage pipeline. It starts with Request Ingress and Parsing, which normalize the incoming request. Then Preprocessing decodes and tokenizes the input. That feeds into our ML Detection Engine — the PyTorch model running on CUDA — which classifies the input and produces a confidence score. From there, the request is enriched with Threat Intelligence, Session Manager, and Device Profiler data running in parallel. The Behavior Engine scores anomalies — things like unusually high request rates or endpoint enumeration. All of these signals are fused together by the Risk Engine into a single capped risk score, which the Decision Engine converts into a verdict — Allow, Monitor, Rate Limit, or Block. The Policy Engine then applies hard overrides for especially critical cases. Finally, flagged requests are persisted by the Incident and Memory Service, aggregated by Analytics and Reporting, and surfaced to a security analyst through our Dashboard.

***Presenter tip:** Point directly at the diagram while narrating — walk left to right along the top row, then left to right along the bottom row, following the connector arrow.*

SLIDE 12 — Methodology (contd.) — Approach and Steps

Our development approach follows these steps in order. We begin with data collection — gathering labeled benign, SQL Injection, and XSS samples from Kaggle and OWASP payload references. Next we clean and label the data into a single unified schema with three classes. We preprocess the text by decoding URL and HTML encoding, normalizing case, and tokenizing at the character level. Then we design and train our character-level CNN/BiLSTM model in PyTorch on a CUDA-enabled GPU. In parallel, we build the context enrichment modules — threat intelligence, session manager, and device profiler. We then implement the risk, decision, and policy logic that converts all of this into a final verdict. System integration wraps everything into a Flask gateway service connected to MongoDB or PostgreSQL. Finally, we build the React-based dashboard for incident review and verdict override.

SLIDE 13 — Methodology (contd.) — Use Case Description

There are three actors in our system. The End User, or an upstream application in middleware mode, submits requests that trigger the 'Process Request' use case — this internally runs Detect Payload, Enrich Context, Score Risk, and Decide Verdict. The Security Analyst interacts with the 'Review Incidents' use case through our dashboard, where they can replay a flagged request and confirm or override its verdict. And the Developer performs an offline 'Train Model' use case — loading the dataset, preprocessing text, training the neural network on GPU, and evaluating the model. The trained model artifact is what connects this offline training workflow to the live gateway operation and the dashboard review process.

6. Resources and Requirements

SLIDE 14 — Resources/Requirements

On the hardware side, a standard laptop or PC with at least an Intel i5 and 8GB RAM is the minimum requirement. However, because our detection core is a PyTorch deep learning model, we require a CUDA-enabled NVIDIA GPU for training and inference — we recommend an Intel i7 with 16GB RAM for running the gateway and dashboard concurrently. On the software side, we use Python 3.10 or later, with PyTorch in its CUDA-enabled build, along with pandas and numpy for data handling. Flask powers our backend and REST API, with Flask-Caching handling server-side session state. React powers our dashboard, with node-cache for lightweight client-side UI caching. We use MongoDB or PostgreSQL for persistent storage of incidents, sessions, and devices. For development, we use Git and GitHub for version control, VS Code and Jupyter Notebook for development, Postman for API testing, and pytest for unit testing. Our datasets come from Kaggle's SQL Injection and XSS datasets.

7. Expected Outcomes

SLIDE 15 — Expected Outcomes

By the end of this project, we expect to deliver a trained PyTorch deep learning model that classifies input as benign, SQL Injection, or XSS, targeting ninety percent or higher accuracy on held-out test data. We expect a fully integrated fourteen-stage gateway pipeline — not just a standalone classifier, but the complete system covering parsing, context enrichment, risk scoring, decisioning, and policy enforcement. We expect a working incident logging system with an analyst-facing dashboard for reviewing and overriding verdicts. We'll produce a clear evaluation report with accuracy, precision, recall, F1-score, and a confusion matrix, along with confirmation that training was GPU-accelerated. And finally, a complete project report and a live demonstration of the end-to-end gateway.

8. Applications

SLIDE 16 — Applications

This system has several real-world applications. It can be deployed as an actual Web Application Firewall layer — either as a reverse proxy in front of an application, or as middleware integrated directly into one. It's particularly useful for small to medium web applications and startups that need adaptive request protection but don't have a dedicated security team. It also serves as an educational reference architecture, showing how deep learning-based detection can be integrated into a full security operations pipeline — not just a model in isolation. And because our detection approach works at the character level, it generalizes well to other text-based attacks beyond SQLi and XSS, such as command injection, NoSQL injection, or CSRF, within the same pipeline.

9. References

SLIDE 17 — References

Our references include five research papers — Angula and Hashiyana's comparison of classical classifiers, Aburashed and colleagues' 2024 study on ANN-based detection, Krishna, Oluoch and Kim's 2025 hybrid approach, the Scientific Reports 2024 deep learning paper using the HTTP CSIC dataset, and the UniEmbed paper on unified embedding-based detection. We've also referenced the OWASP Top Ten for grounding the problem's real-world relevance, and Kaggle for our SQL Injection and XSS training datasets.

Thank you — we're happy to take any questions.

Presenter tip: End confidently and pause — don't rush into questions. Let the panel lead.